DEVOPS CAPSTONE PROJECT

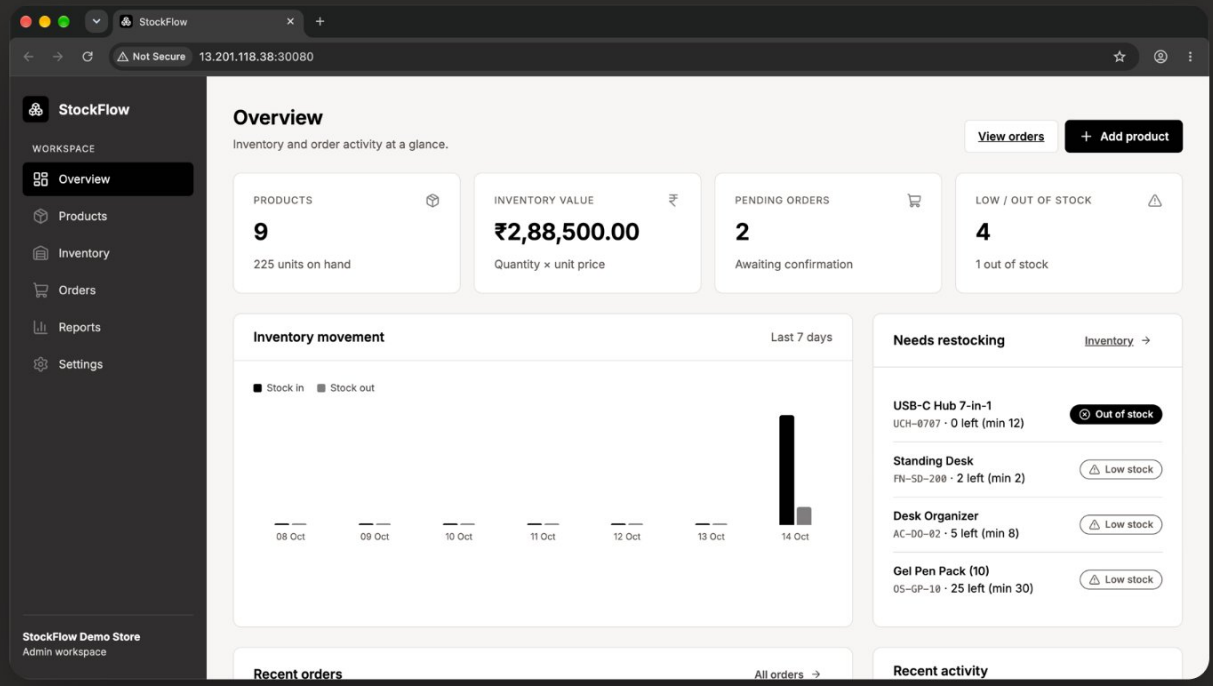
StockFlow

Inventory and order management, taken from a Git commit to a live, monitored app on AWS EKS.

Sambhav D Bohra

Batch B · 24bcs10090

`github.com/sambhavbohra/StockFlow---DevOps-Project`



THE APPLICATION

A shop's products, stock and orders in one place

Products and stock

Every stock change is a recorded movement.

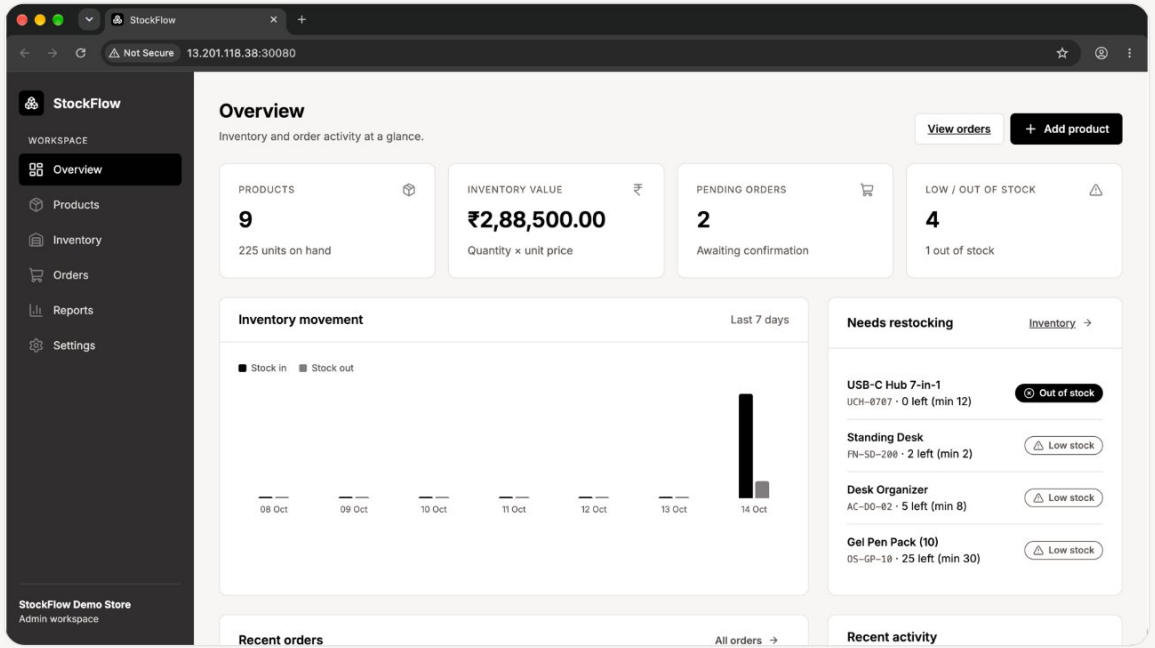
Orders

Pending → Confirmed → Shipped → Delivered. Stock drops only on confirm, under row locks.

Dashboard and reports

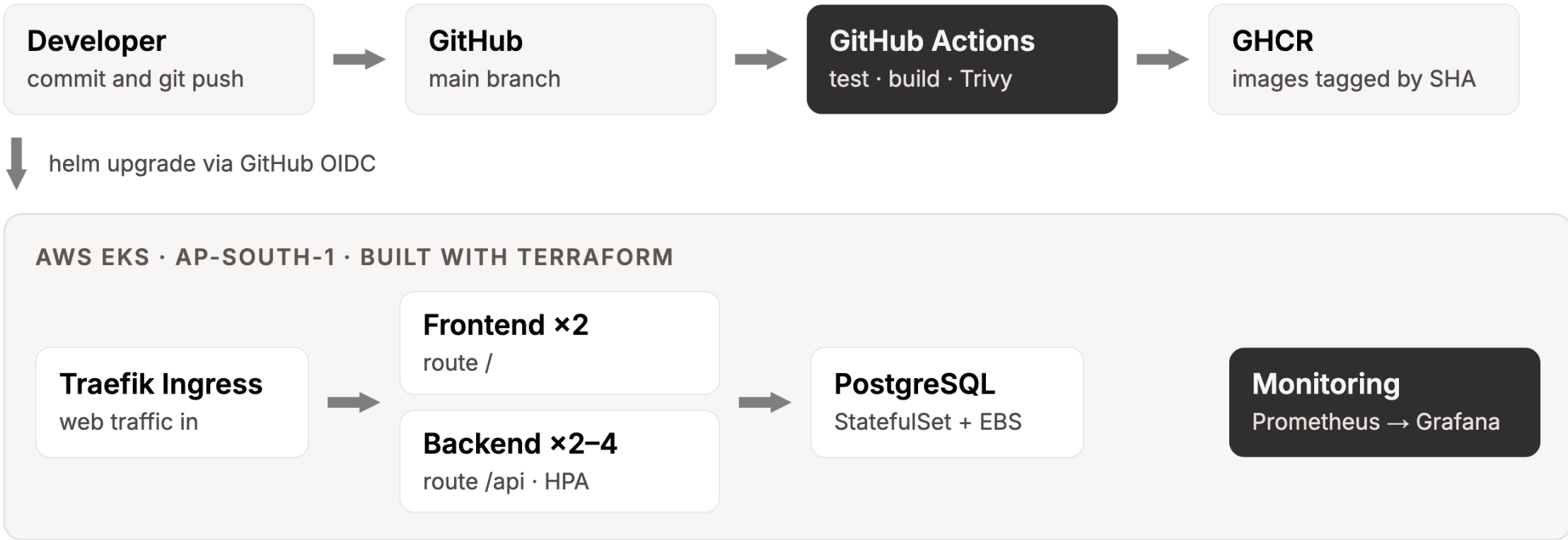
Live KPIs, low-stock alerts and order reports.

React + Vite | FastAPI | PostgreSQL | SQLAlchemy + Alembic | 41 pytest tests



ARCHITECTURE

From a commit to a running cluster



CI/CD PIPELINE

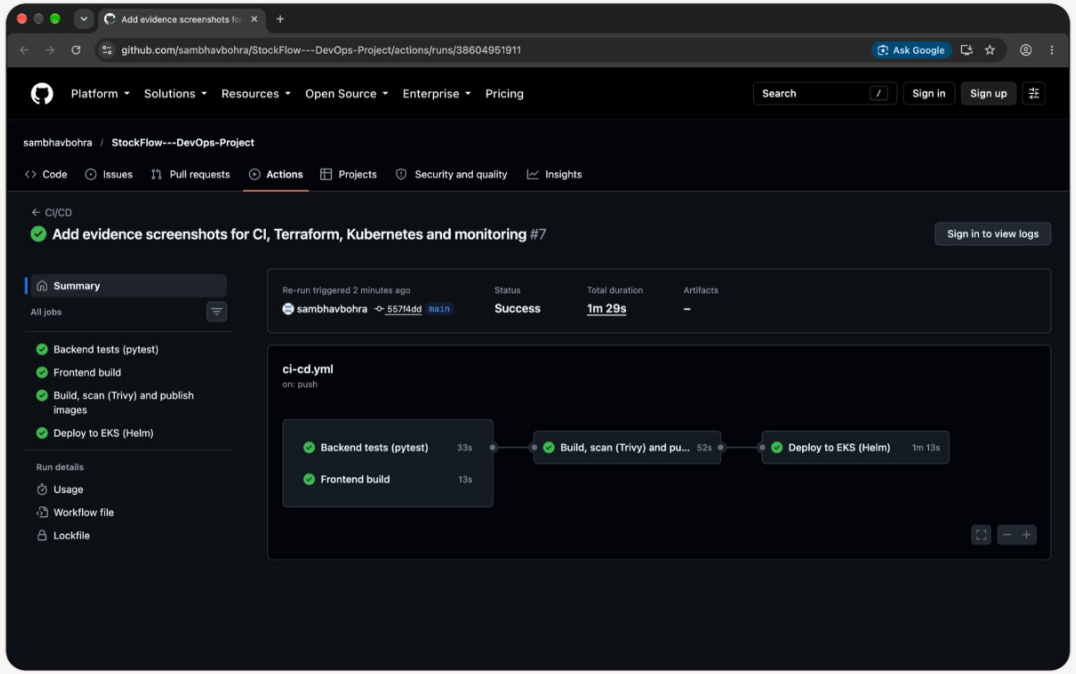
Every push to main is tested, scanned and deployed

1. **Test**
   pytest on a throwaway PostgreSQL, plus the frontend build. A failing test stops everything.

2. **Build**
   Two non-root Docker images, tagged with the Git commit SHA, never "latest".

3. **Scan and publish**
   Trivy blocks fixable HIGH and CRITICAL CVEs, then the images go to GHCR.

4. **Deploy**
   GitHub OIDC → AWS role → helm upgrade on EKS. No AWS keys stored; zero-downtime rollout.



DEVSECOPS

Security is built into the pipeline

Trivy security gate

Fails the build on fixable HIGH or CRITICAL CVEs.

Pinned supply chain

Actions pinned to commit SHAs; Trivy checksum-verified.

Non-root containers

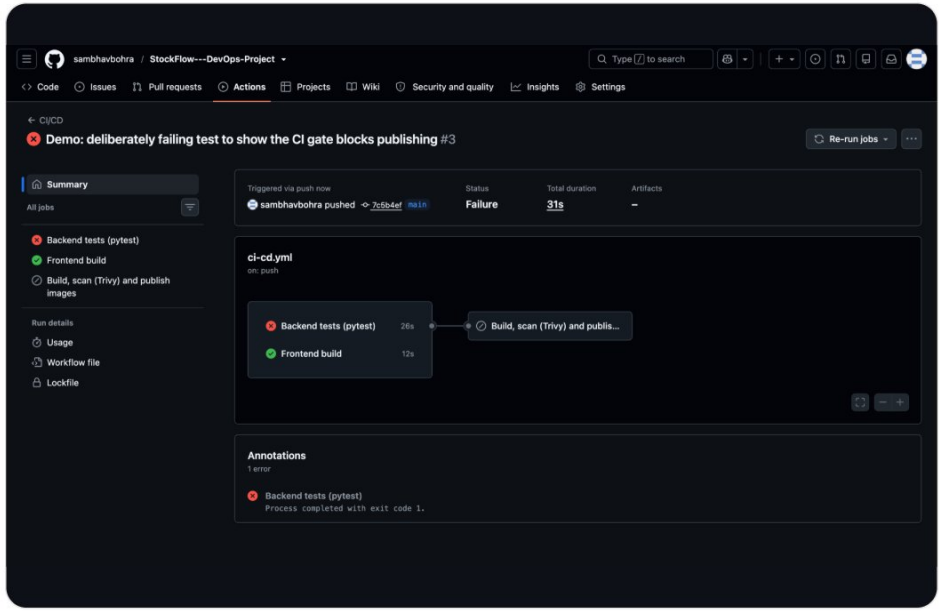
Backend runs as UID 10001, frontend as UID 101.

No secrets in Git

Passwords live only in Kubernetes Secrets; CI logs in with OIDC.

REAL FINDING

Trivy flagged HIGH CVEs inside pip's bundled libraries. Removing pip from the image fixed them all.



A failing test blocks publishing: no image is built.

INFRASTRUCTURE AS CODE

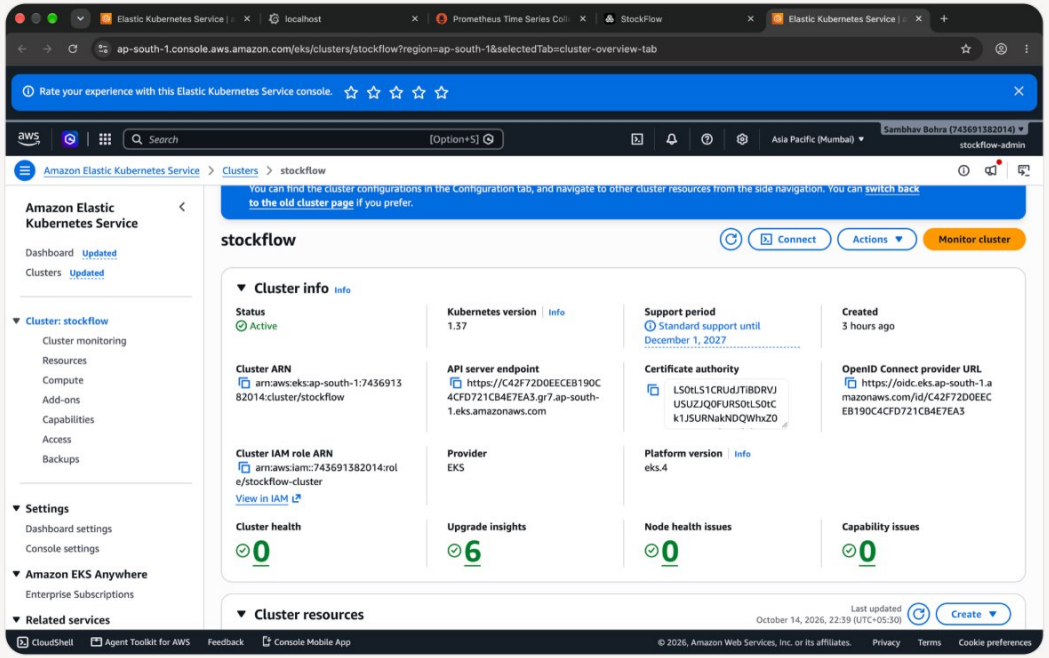
AWS is built with Terraform, not clicks

Part	What Terraform creates
Network	VPC, 2 public subnets in 2 zones, no NAT gateway
Cluster	EKS, Kubernetes 1.37, standard support pinned
Nodes	2 × c7i-flex.large on Spot
Add-ons	VPC CNI, CoreDNS, EBS CSI, metrics-server
CI access	GitHub OIDC role, limited to one namespace

≈ $0.20
per hour while running

29
resources, one command

Mumbai
region ap-south-1



The EKS cluster, Active in the AWS console.

KUBERNETES + HELM

One Helm chart runs the whole app

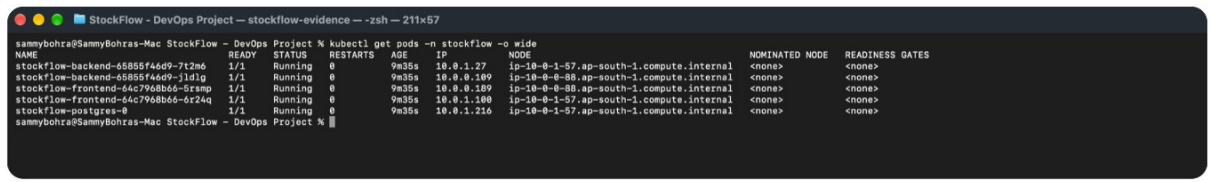
Deployments
2+ replicas each, health probes, CPU and memory limits

Services + Ingress
ClusterIP inside; Traefik routes / and /api

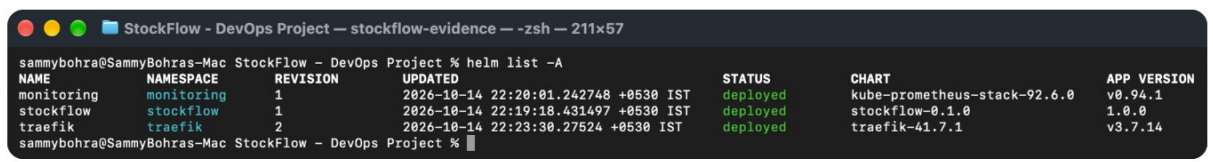
Autoscaling
HPA scales the backend from 2 to 4 pods at 70% CPU

Database
PostgreSQL StatefulSet on a 1 GiB EBS volume

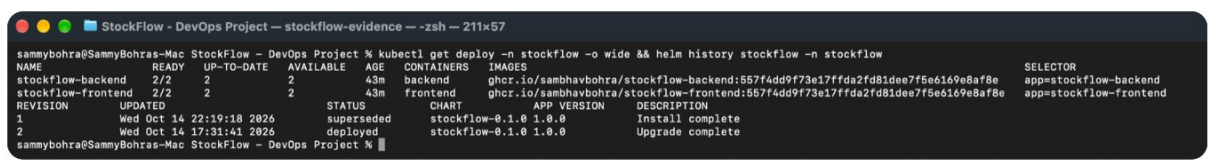
`kubectl get pods -n stockflow`



`helm list -A`

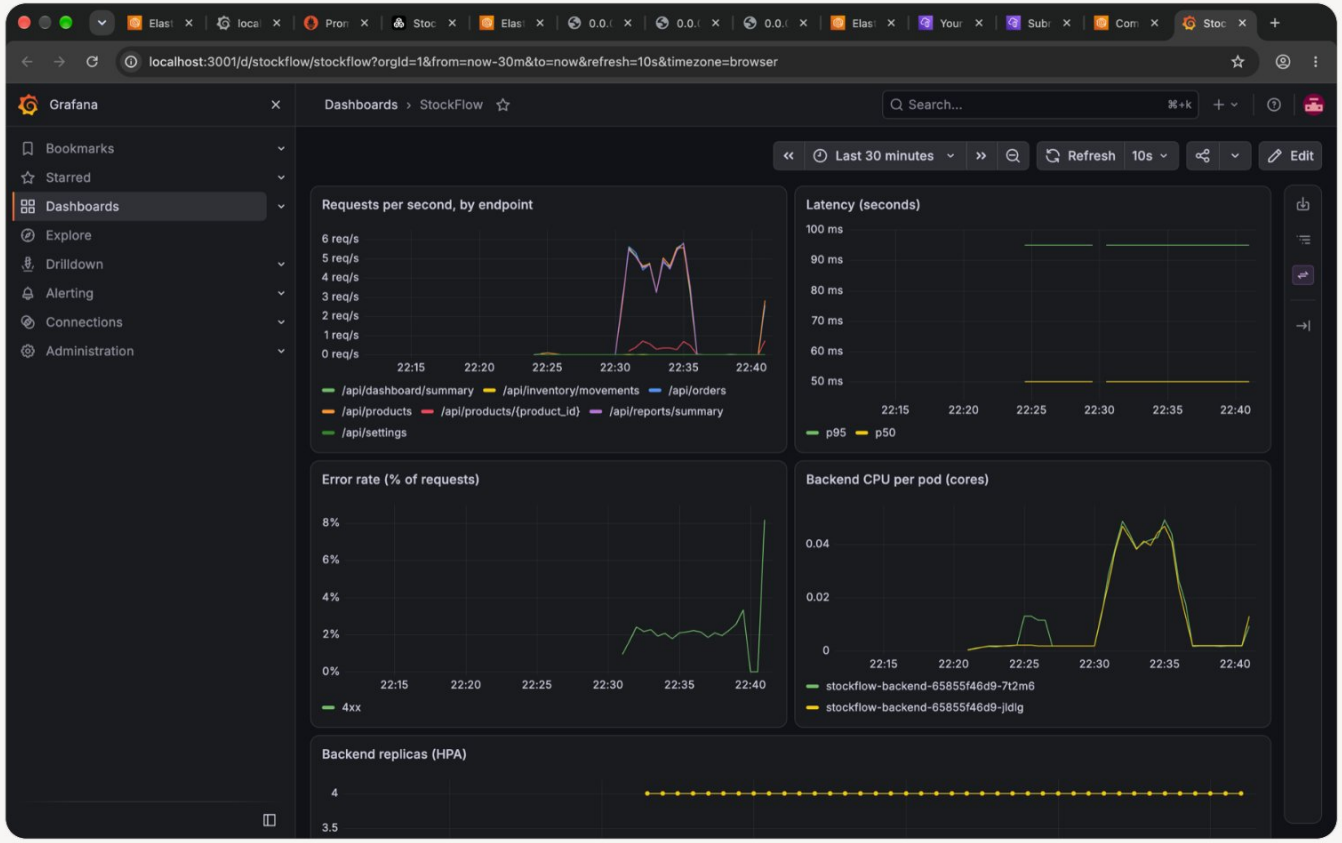


`after a push: new SHA, Helm revision 2`



OBSERVABILITY

Live metrics from the running app



1 · /metrics
FastAPI exports request counts and latency

2 · ServiceMonitor
Tells Prometheus to scrape every 15 seconds

3 · Prometheus
Both backend targets UP (2 of 2)

4 · Grafana
Requests/s, p50 and p95 latency, error %, CPU per pod, HPA replicas

TROUBLESHOOTING

Real problems I solved on AWS

Problem	Cause	Fix
Spot nodes would not launch	Free plan allows only Free Tier instance types	Switched to c7i-flex.large: 2 vCPU, 4 GiB
No load balancer created	AWS blocked load balancers on this account	Traefik on NodePort 30080 plus one firewall rule
Pipeline deploy denied	GitHub's OIDC token now names the repo by ID	Found it in CloudTrail; updated the IAM trust policy
Backend restarted on first install	Migrations started before PostgreSQL was ready	Init container waits for the database
Spot server reclaimed mid-demo	AWS can take Spot capacity back	Kubernetes moved the pods; the app stayed up

RESULTS

Built, shipped, monitored, then torn down

41
automated tests, run on every push

4
pipeline stages, ending in a deploy

29
AWS resources created and destroyed

0
resources left running after teardown

CODE AND EVIDENCE

github.com/sambhavbohra/StockFlow---DevOps-Project

DEMO VIDEO

loom.com/share/2fbe9b176f9a4030b375d44de9c86bab

Thank you

Sambhav D Bohra · Batch B · 24bcs10090